

PRACTICAL WEEK-12

1 . Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Aim : To implement producer-consumer synchronization using POSIX threads and counting semaphores, and compare throughput for different buffer sizes.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 1000000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i + 1;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);
    }
}
```

^G Help ^O Write Out ^W Where Is ^P
^X Exit ^R Read File ^_ Replace ^L

```
    sem_wait(&full);

    pthread_mutex_lock(&mutex);

    sum += buffer[out];
    out = (out + 1) % buffer_size;

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);
}

printf("Consumer processed %d items.\n", ITEMS);
printf("Consumer checksum: %lld\n", sum);

return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0)
    {
        printf("Buffer size must be greater than 0.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("malloc");
        return 1;
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

GNU nano 7.2

```
sem_init(&empty, 0, buffer_size);
sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);

printf("Producer-Consumer Problem\n");
printf("-----\n");
printf("Buffer size : %d\n", buffer_size);
printf("Items      : %d\n\n", ITEMS);

clock_gettime(CLOCK_MONOTONIC, &start);

pthread_create(&producer_thread,
              NULL,
              producer,
              NULL);

pthread_create(&consumer_thread,
              NULL,
              consumer,
              NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

double throughput = ITEMS / elapsed;

printf("\nSynchronization completed successfully.\n");
printf("Execution time : %.6f seconds\n", elapsed);
printf("Throughput      : %.2f items/second\n", throughput);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano producer_consumer.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -pthread producer_consumer.c -o producer_consumer
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 10
Producer-Consumer Problem
-----
Buffer size : 10
Items       : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 0.935073 seconds
Throughput    : 106897.74 items/second
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 1
Producer-Consumer Problem
-----
Buffer size : 1
Items       : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 4.511384 seconds
Throughput    : 22165.16 items/second
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 5
Producer-Consumer Problem
-----
Buffer size : 5
Items       : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 1.017715 seconds
Throughput    : 98546.90 items/second

```

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 10
Producer-Consumer Problem
-----
Buffer size : 10
Items       : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 0.873530 seconds
Throughput    : 114478.04 items/second
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 50
Producer-Consumer Problem
-----
Buffer size : 50
Items       : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 0.179109 seconds
Throughput    : 558320.70 items/second

```

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./producer_consumer 100
Producer-Consumer Problem
-----
Buffer size : 100
Items      : 100000

Consumer processed 100000 items.
Consumer checksum: 5000050000

Synchronization completed successfully.
Execution time : 0.097906 seconds
Throughput    : 1021392.19 items/second
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |
```

Observation:

The Producer-Consumer problem was implemented using POSIX threads and counting semaphores. Different buffer sizes were tested, and the execution time and throughput changed with the buffer size. The consumer successfully processed all 100000 items with the expected checksum.

Result:

The program successfully achieved synchronization between the producer and consumer. All items were processed correctly without data loss, demonstrating that counting semaphores can effectively control access to the shared buffer and support correct synchronization.

2 . Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Aim : To demonstrate a deadlock between multiple POSIX threads competing for shared mutex resources and prevent the deadlock by acquiring resources in a fixed order.

Program :

```
}  
  
void *thread2_function(void *arg)  
{  
    (void)arg;  
  
    printf("Thread 2: Trying to acquire Resource 2\n");  
    pthread_mutex_lock(&resource2);  
  
    printf("Thread 2: Acquired Resource 2\n");  
    sleep(1);  
  
    printf("Thread 2: Trying to acquire Resource 1\n");  
    pthread_mutex_lock(&resource1);  
  
    printf("Thread 2: Acquired Resource 1\n");  
  
    pthread_mutex_unlock(&resource1);  
    pthread_mutex_unlock(&resource2);  
  
    return NULL;  
}  
  
int main()  
{  
    pthread_t thread1, thread2;  
  
    pthread_mutex_init(&resource1, NULL);  
    pthread_mutex_init(&resource2, NULL);  
  
    printf("Deadlock Demonstration\n");  
    printf("-----\n");  
  
    pthread_create(&thread1, NULL, thread1_function, NULL);  
    pthread_create(&thread2, NULL, thread2_function, NULL);  
  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
  
    pthread_mutex_destroy(&resource1);  
    pthread_mutex_destroy(&resource2);  
  
    printf("Program completed.\n");  
  
    return 0;  
}
```

```
^G Help  
^X Exit
```

```
^O Write Out  
^R Read File
```

```
^W Where Is  
^_ Replace
```

```
^K Cut  
^U Paste
```

```
^T  
^J
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    (void)arg;

    printf("Thread 1: Acquiring Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Acquired Resource 1\n");
    sleep(1);

    printf("Thread 1: Acquiring Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired Resource 2\n");

    printf("Thread 1: Using both resources\n");
    sleep(1);

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released\n");

    return NULL;
}

void *thread2_function(void *arg)
{
    (void)arg;

    /*
     * Resource ordering:
     * Resource 1 must always be acquired before Resource 2.
     */

    printf("Thread 2: Acquiring Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1\n");
    sleep(1);

    printf("Thread 2: Acquiring Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2\n");
```

```
* Resource ordering:
* Resource 1 must always be acquired before Resource 2.
*/

printf("Thread 2: Acquiring Resource 1\n");
pthread_mutex_lock(&resource1);

printf("Thread 2: Acquired Resource 1\n");
sleep(1);

printf("Thread 2: Acquiring Resource 2\n");
pthread_mutex_lock(&resource2);

printf("Thread 2: Acquired Resource 2\n");

printf("Thread 2: Using both resources\n");
sleep(1);

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

printf("Thread 2: Resources released\n");

return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("Deadlock Prevention Using Resource Ordering\n");
    printf("-----\n");

    printf("Resource order: Resource 1 -> Resource 2\n\n");

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nAll threads completed successfully.\n");
    printf("Deadlock prevented using resource ordering.\n");

    return 0;
}
```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano deadlock.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -pthread deadlock.c -o deadlock
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./deadlock
Deadlock Demonstration
-----
Thread 1: Trying to acquire Resource 1
Thread 2: Trying to acquire Resource 2
Thread 2: Acquired Resource 2
Thread 1: Acquired Resource 1
Thread 2: Trying to acquire Resource 1
Thread 1: Trying to acquire Resource 2
^C

jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano deadlock_prevention.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra -pthread deadlock_prevention.c -o deadlock_prevention
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./deadlock_prevention
Deadlock Prevention Using Resource Ordering
-----
Resource order: Resource 1 -> Resource 2

Thread 2: Acquiring Resource 1
Thread 2: Acquired Resource 1
Thread 1: Acquiring Resource 1
Thread 2: Acquiring Resource 2
Thread 2: Acquired Resource 2
Thread 2: Using both resources
Thread 2: Resources released
Thread 1: Acquired Resource 1
Thread 1: Acquiring Resource 2
Thread 1: Acquired Resource 2
Thread 1: Using both resources
Thread 1: Resources released

All threads completed successfully.
Deadlock prevented using resource ordering.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$
```

Observation:

A deadlock was demonstrated when multiple threads acquired shared resources in different orders. Each thread held one resource while waiting for another, causing the threads to remain blocked. A fixed resource-ordering strategy was then implemented to avoid this situation.

Result:

The deadlock was successfully prevented by making all threads acquire resources in the same order. All threads completed their execution and released the resources successfully, demonstrating an effective deadlock-prevention technique.